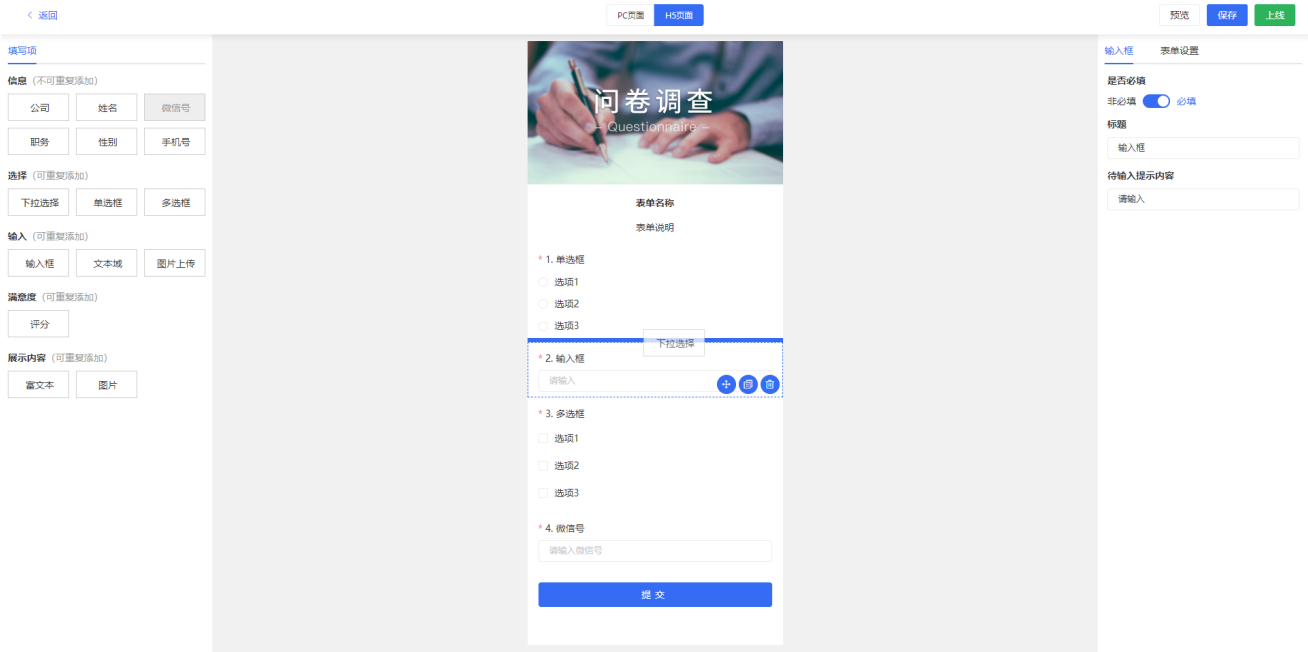
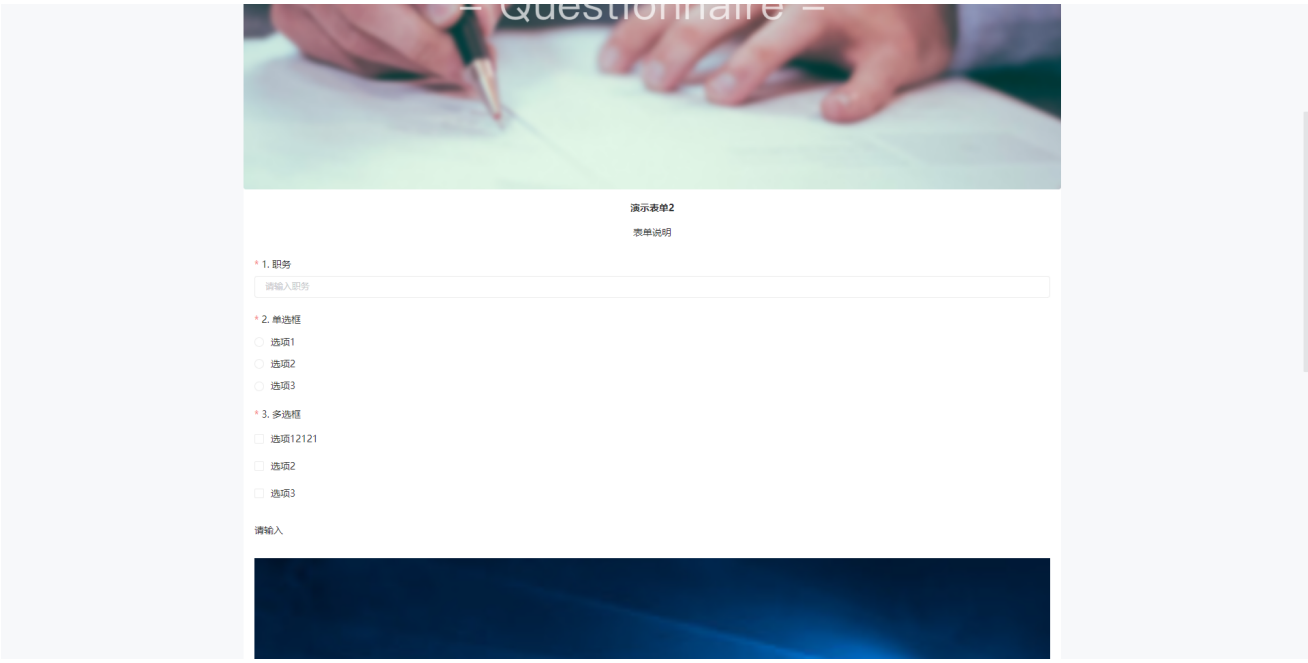
- 功能描述
  - 通过拖拽或点击生成自定义表单
  - 保存后生成表单的链接，访问此链接可填写并提交内容
  - 提交的内容在系统中可以获取到并做一些数据分析
- 演示
  - 创建



  - 预览
    - pc：



    - 手机：

10:32 5G 76

t-marketing.qiankebao.com.cn

扫描立即领取
价值648元的
知了标讯VIP会员福利

表单sasa名称

表单说sasasa明

* 公司

请输入公司

* 输入框

请输入

* 文本域qwqwq

请输入wqwqwq

- 宏观思路
  - 核心思想
    - 数据驱动视图
      - 拖拽表单元素、创建自定义的表单、配置表单、保存表单、获取已保存表单.....都是在操作数据
    - 详细描述
      - 创建时，生成了两条数据，一条存的是表单整体配置，一条存的是表单中有哪些元素和表单元素对应的配置
      - 保存后把这两条数据存到数据库，并生成访问链接，实际上这个链接主要由当前系统的另一个页面的路由地址和当前表单的id组成
      - 预览表单就是跳转当前系统的另一个页面，并根据表单id获取存到数据库中的表单配置数据、表单结构数据，并还原成表单结构
    - 如图所示

```
sunjingfeng, 3个月前 | 1 author (sunjingfeng)
export default [
  // 基本设置
  {
    title: '基本设置',
    value: [
      // 头图
      {
        key: 'headImage',
        value: {
          show: true,
          imageUrl: ''
        }
      },
      // 表单名称
      {
        key: 'formName',
        value: {
          value: '表单名称'
        }
      },
      // 表单说明
      {
        key: 'formInstruction',
        value: {
          show: true,
          value: '表单说明'
        }
      },
      // 按钮文案
      {
        key: 'formButton',
        value: {
          value: '提交'
        }
      },
      // 标题位置
      {
        key: 'titlePosition',
        value: {      sunjingfeng, 6个月前 • feat: change
          value: 'top'
        }
      },
      // 是否展示序号
      {
        key: 'showNumber',
        value: {
          value: true
```

初始的表单配置，每个对象对应一条配置

- 表单创建
  - 表单配置
    - 表单整体配置是一个对象，每条配置项是对象中的一个键值对
  - 表单结构
    - 整体结构是一个数组，每个表单元素是数组中的一个对象
    - 拖拽时获取表单元素对象，释放时获取位置，将目标放到数组中的对应位置
  - 保存
    - 将配置对象、表单元素数组存到数据库
    - 如图所示

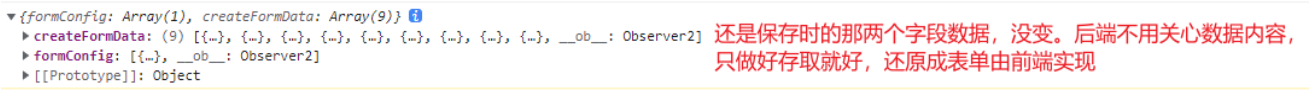
```
▼{formConfig: Array(1), createFormData: Array(4)}
  ▼createFormData: Array(4)
    ▶0: {iconClassList: Array(2), title: '输入框', type: 'single-input', mouseon: '', initialSubmitDataValue: Array(1), …}
    ▶1: {iconClassList: Array(2), title: '文本域', type: 'multi-input', mouseon: '', initialSubmitDataValue: Array(1), …}
    ▶2: {iconClassList: Array(2), title: '图片上传', type: 'image-upload', mouseon: '', initialSubmitDataValue: Array(1), …}
    ▶3: {iconClassList: Array(2), title: '评分', type: 'score', mouseon: '', initialSubmitDataValue: Array(1), …}
     length: 4
    ▶[[Prototype]]: Array(0)
  ▼formConfig: Array(1)
    ▼0:
       title: "基本设置"
      ▼value: Array(6)
        ▶0: {key: 'headImage', value: {…}}
        ▶1: {key: 'formName', value: {…}}
        ▶2: {key: 'formInstruction', value: {…}}
        ▶3: {key: 'formButton', value: {…}}
        ▶4: {key: 'titlePosition', value: {…}}
        ▶5: {key: 'showNumber', value: {…}}
         length: 6
        ▶[[Prototype]]: Array(0)
      ▶[[Prototype]]: Object
     length: 1
    ▶[[Prototype]]: Array(0)
  ▶[[Prototype]]: Object
>
```

表单结构数据

表单配置数据

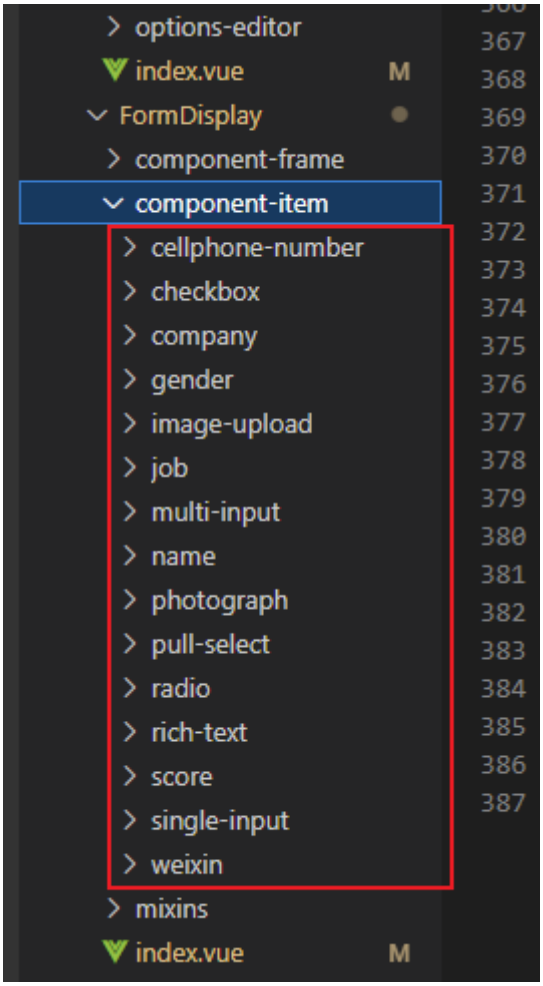
- 说明：
  - 为了能方便看清楚数据，图片中的数据做了简化处理。实际这里传给后端的是将两条数据整合起来的JSON字符串和一些其他业务数据
  - 后端不需要关心这个JOSN字符串的内容，只管存取

- 表单加载和提交
  - 加载
    - 获取之前保存到数据库中的配置对象、表单元素数组，利用前端代码生成表单
    - 如图所示



  - 提交
    - 表单相关数据（配置对象、表单元素数组）和用户提交内容都需要传给后端
    - 用户提交的内容的传递格式需要根据业务需求，前后端沟通后制定格式
    - 如图所示

- 实现细节
  - 拖拽相关
    - 表单创建
      - 为保证灵活性，使用了原生draggable相关接口
      - 文档：HTML 拖放 API、拖拽操作
    - 选择题选项顺序调整
      - 功能较简单，使用了插件
      - 文档：vue.draggable中文文档
  - 效率开发
    - 代码复用
      - 组件复用
        - 说明
          - 合理的拆分组件，能解耦复杂页面，从而提高灵活性、可读性、代码复用性等
          - 拆分组件语法依据技术栈不同而不同，属于基本语法，可查阅相应技术栈文档
        - 例子
          - 每个可拖拽的元素都是组件
            - 如图所示




```
// 自动注册component-item文件夹组件
function fn() {
  const componentItemComponents = {}
  const componentItems = import.meta.glob('./component-item/**/*.vue', { eager: true })
  Object.keys(componentItems).forEach(key => {
    key.replace(/\.\/component-item\/(\S+)\//, (match, p1) => {
      componentItemComponents[p1] = componentItems[key].default
    })      sunjingfeng, 6个月前 • feat: change
  })
  return componentItemComponents
}
```

```
    }"
    :draggable="formItemBoxDraggable"
    @dragstart="$emit('dragstart', item, 'form')"
    @dragend="$emit('dragend')"
    @click="$emit('clickFormItem', item)">
    <component
      :is="item.type"
      ref="component"
      :initialSubmitDataValue="item.initialSubmitDataValue || []"
      :config="item.config"
      :number="index + 1"
      :formConfig="formConfig"
      :searchFormConfig="searchFormConfig" />
    <div v-if="model === 'create'" class="block">        sunjingfeng, 4个月前 • feat: change
      <div class="control-box" v-if="currentFocusItem === item">
        <el-tooltip
          v-if="createFormData.length > 1"
          class="drag"
          effect="dark"
          content="排序"
          placement="top">
```

- 创建表单时中间的表单演示部分、预览时的表单、用户提交的表单，统一封装成一个组件
  - 如图所示

```
                draggable="true"
                @dragstart="dragstart(item2, 'sidebar')"
                @click="clickSidebarItem(item2)">
                <i :class="[...item2.iconClassList]" />
                <span>{{ item2.title }}</span>
              </div>
            </template>
          </div>
        </div>
      </el-tab-pane>
      <!-- <el-tab-pane label="展示项" name="2">展示项内容</el-tab-pane> -->
    </el-tabs>
  </div>
  <div class="form" @drop="drop">
    <FormDisplay
      :h5="h5"
      :formConfig="formConfig"
      :createFormData="createFormData"
      :placeholdeActive="placeholdeActive"
      :currentFocusItem="currentFocusItem"
      :formItemBoxDraggable="formItemBoxDraggable"
      @formItemBoxDraggableChange="formItemBoxDraggableChange"
      @dragstart="dragstart"
      @dragend="dragend"
      @clickFormItem="clickFormItem"
      @copyFormItem="copyFormItem"
      @deleteFormItem="deleteFormItem" />
  </div>
  <div class="right-sidebar">
    <el-tabs v-model="rightSidebarTab">
      <el-tab-pane v-if="currentFocusItem" :label="currentFocusItem.title" name="1">
        <!-- key拼接createFormData.indexOf(currentFocusItem)的作用，是保证切换不同组件的相同配置项时，component-config-item会重新渲染 -->
        <component
          v-for="item in currentFocusItem.config"
          :key="item.type + createFormData.indexOf(currentFocusItem)"
          :is="item.type"
          v-model="item.value"
```

创建表单的时候在页面中间做演示用

```
sunjingfeng, 3个月前 | 2 authors (sunjingfeng and others)
<template>
  <PreviewFrame class="form-preview" v-slot="{ response }">
    <FormDisplay
      model="static"
      :createFormData="createFormData(response)"
      :formConfig="formConfig(response)" />
  </PreviewFrame>
</template>
```

预览的时候

```vue
<template>
  <div class="form-submit map-full-screen">
    <FormDisplay
      v-if="showForm"
      model="submit"
      :params="{
        ...$route.query,
        answerPeriod: answerPeriod,
        formDataId: uuid
      }"
      :createFormData="createFormData"
      :formConfig="formConfig"
      @success="showForm = false" />
    <el-result v-else icon="success" title="提交表单成功" />
  </div>
</template>
```

用户提交的时候

- 部分逻辑复用
  - 说明
    - 根据技术栈不同语法也不同，但都会提供部分逻辑复用的语法，需查阅相应文档
  - 文档
    - vue2：混入
    - vue3：组合式函数
    - react：hook
  - 本功能使用的部分混入，如图所示

```
sunjingfeng, 3个月前 | 1 author (sunjingfeng)
import ConfigFrame from '@/components/FormCreater/config-frame/index.vue'

export default {
  components: {
    ConfigFrame
  },
  props: {
    // 配置项的key
    configKey: {
      type: String,
      required: true
    },
    // 初始配置项值
    formConfigObj: {
      type: Object,
      required: true
    }
  },
  data() {
    return {
      innerFormConfigObj: {} // 配置项值
    }
  },
  watch: {
    // 更新表单设置
    innerFormConfigObj: {
      handler(newVal) {
        this.$emit('updateFormConfigObj', { key: this.configKey, value: newVal })
      },
      deep: true
    }
  },
  created() {
    this.innerFormConfigObj = this.formConfigObj
  },
  methods: {
    // switchValue改变
    switchValueChange(value) {
      this.innerFormConfigObj.show = value
    }
  }
}
```

将多个组件的部分逻辑抽离

```
<template>
  <ConfigFrame class="form-name" title="表单名称">
    <el-input
      type="textarea"
      :rows="2"
      placeholder="请输入"
      v-model="innerFormConfigObj.value"
      maxlength="20"
      show-word-limit
      resize="none" />
  </ConfigFrame>
</template>

<script>
import mixin from '@/components/FormCreater/mixins/form-config-item-mixin'

export default {
  name: 'FormName',
  mixins: [mixin]
}
</script>

<style lang="scss" scoped>
.form-name {    Do not use empty rulesets
}
</style>
```

每个组件中仅需两行代码便可复用混入中的全部逻辑

- 重复操作自动化

- 自动注册组件
  - 说明
    - 根据打包工具不同语法也不同，需参考打包工具文档
    - 这是个批量读取文件并执行操作的接口，不仅可以注册组件，当文件目录庞大又需要大量引入的时候都可以用这个接口
  - 文档
    - webpack：require.context()
    - vite：import.meta.glob()
- 根据数据自动渲染相应组件
  - 说明
    - 每条配置项都拆分成了组件，根据当前表单元素的配置项内容批量加载组件
  - 文档：component
  - 如图所示

```vue
<div class="right-sidebar">
  <el-tabs v-model="rightSidebarTab">
    <el-tab-pane v-if="currentFocusItem" :label="currentFocusItem.title" name="1">
      <!-- key拼接createFormData.indexOf(currentFocusItem)的作用，是保证切换不同组件的相同配置项时，component-config-item会重新渲染 -->
      <component
        v-for="item in currentFocusItem.config"
        :key="item.type + createFormData.indexOf(currentFocusItem)"
        :is="item.type"
        v-model="item.value"
        :config="item" />
    </el-tab-pane>
    <el-tab-pane label="表单设置" name="2">
      <!-- key的存在保证编辑时把formConfig整体替换后，v-model绑定的是新formConfig的值。因此this.id的赋值要在formConfig替换为新值之后-->
      <el-collapse :key="id" v-model="formConfigCollapse">
        <el-collapse-item
          v-for="(item, index) in formConfig"
          :key="index"
          :title="item.title"
          name="1">
          <component
            v-for="(item2, index2) in item.value"
            :key="index2"
            :is="item2.key"
            @updateFormConfigObj="updateFormConfigObj"
            :configKey="item2.key"
            :formConfigObj="item2.value" />
        </el-collapse-item>
      </el-collapse>
    </el-tab-pane>
  </el-tabs>
</div>
```